



Rossmann Store Sales Forecasting

IT402 – Applied Forecasting Methods

Group 15

Utsav Solanki (202303026)
Pranav Bavadiya (202301458)
Aryan Rathod (202301164)
Rudra Chauhan (202301173)

Instructor: Dr. Pritam Anand
Institute: DAIICT

Academic Year: 2025 - 2026
Submission Date: May 7, 2026

Contents

Abstract	3
1 Introduction	4
1.1 Background and Motivation	4
1.1.1 Context of the Forecasting Problem	4
1.1.2 Relevance and Importance of Sales Forecasting	4
1.1.3 Associated Challenges	5
1.2 Problem Statement	6
1.3 Objectives	6
2 Data Source and Pre processing	7
2.1 Data Source	7
2.2 Data Characteristics	7
2.2.1 Variables and Descriptions	7
2.2.2 Data Size and Frequency	8
2.3 Data Pre processing	8
2.3.1 Merging Data Sources	8
2.3.2 Handling Missing Values	9
2.3.3 Categorical Encoding	9
2.3.4 Feature Engineering	9
2.3.5 Data Aggregation (for Aggregated Sales Analysis)	10
2.4 Exploratory Data Analysis	10
2.4.1 Stationarity Testing	10
2.4.2 STL Decomposition	11
3 Methodology	14
3.1 Forecasting Horizon and Task Definition	14
3.2 Input Output Structure	14
3.3 Train Test Split Strategy	15
3.3.1 Time Based Split	15
3.3.2 Validation Approach for Hyperparameter Tuning	16
3.4 Hyperparameter Tuning	16
3.4.1 Classical Time Series Models (ARIMA, SARIMA, ARIMAX, SARI- MAX)	16
3.4.2 Prophet Model	16
3.4.3 Deep Learning Models (RNN, LSTM, GRU, etc.)	17
3.5 Evaluation Metrics	17

3.6	Model Implementations	18
3.6.1	Autoregressive Model (AR)	18
3.6.2	Moving Average Model (MA)	18
3.6.3	ARIMA	18
3.6.4	SARIMA	18
3.6.5	SARIMAX	19
3.6.6	Facebook Prophet	19
3.6.7	TBATS	19
3.6.8	Simple Recurrent Neural Network (SimpleRNN)	19
3.6.9	Long Short Term Memory (LSTM)	19
3.6.10	Gated Recurrent Unit (GRU)	20
3.6.11	Fully Connected Neural Network (Dense Model)	20
3.6.12	Transformer Model	20
3.6.13	Temporal Convolutional Network (TCN)	20
3.6.14	N-HiTS Model	21
3.6.15	PatchTST Model	21
3.6.16	Quantile Regression	21
4	Analysis of Numerical Results	22
4.1	Results: Classical Statistical Models	22
4.2	Results: Aggregated Forecasting	22
4.3	Results: Deep Learning Models	23
4.4	Results: Quantile Regression	23
4.5	Ablation Study	24
4.6	Research Gap Identified	24
5	Converting the Forecast into Actions	25
5.1	Inventory and Stock Management	25
5.2	Staff Scheduling	25
5.3	Promotion Planning and Timing	25
5.4	Supply Chain and Distribution Logistics	25
6	Future Work	26
7	Code and Live Website	26
8	References	27

Abstract

Forecasting sales in retail is crucial for controlling stocks. Accurate forecasts ensure efficient operations with minimum losses. This project aims to predict store sales using the Rossmann Store Sales dataset including sales in 1,115 stores over several years.

Rossmann stores have plenty of factors influencing customer behavior other than time patterns in data. The list of factors includes promotional events, public holidays, school holidays, type of a store, number of competing stores and seasonality.

This project considers two approaches to forecasting – aggregate sales and store-based sales. We used the classic forecasting methods: AutoRegressive, ARIMA and SARIMAX. In addition, we utilized more advanced approaches: Prophet and deep learning (RNN, GRU, LSTM) models for capturing non-linear patterns in data. To receive not only point predictions but also intervals for forecasting, Quantile Regression is also done.

The results showed that SARIMAX and Prophet models perform quite well on Rossmann data, SARIMAX even showing a good accuracy after incorporating external factors. Models for capturing non linear patterns worked effectively but had to be adjusted to work properly. Forecast intervals obtained by Quantile Regression were useful for better business decisions. Overall, the use of both traditional and deep learning methods turned out to be effective in sales forecasting.

1 Introduction

1.1 Background and Motivation

1.1.1 Context of the Forecasting Problem

Rossmann is one of the largest drug store chains in Europe with thousands of outlets in multiple countries. Every store functions in a specific economic environment and has unique issues in terms of operations management. Rossmann Store Sales forecasting is a real world regression problem that consists in estimating the sales of particular stores on a given day.

Retail sales in the real world depend on many factors. For a chain such as Rossmann, accurate knowledge of future sales is essential in order to optimize decisions on stock, promotions and other important aspects of operations management. Estimating the future sales of a store is a necessary step for making important business decisions with wide ranging consequences.

1.1.2 Relevance and Importance of Sales Forecasting

There are a number of business benefits associated with accurate sales forecasting:

1. **Stock Management:** Forecasts allow retail chains to order sufficient quantity of products without risking losses due to oversupply. On the contrary, stock shortages lead to lost sales and dissatisfied customers.
2. **Personnel Scheduling:** With forecasts, retail stores can schedule a sufficient number of employees per working day without unnecessary expenses. At the same time, understaffed stores will suffer from poor customer satisfaction and excessive stress on remaining employees.
3. **Marketing Campaigns and Promotions:** Retailers use forecasts in their marketing strategies. A promotion campaign is most effective when launched during periods of increased traffic.
4. **Budget Planning:** Accurate sales forecasts allow retailers to plan their budget allocation and profit goals in a realistic manner.
5. **Optimization of Supply Chains:** Suppliers and distribution centers use forecasts to determine optimal schedules of product deliveries.

1.1.3 Associated Challenges

Rossmann's sales forecasting is a more complicated problem than time series extrapolation. Several important issues complicate this task:

- **Impact of Promotions:** A store's sales increase sharply on a promotion day. However, the magnitude of the effect depends on the specifics of a promotion itself, season and context of the store. Conventional statistical models tend to underestimate or ignore these factors.
- **Public and School Holidays:** Holiday periods disrupt regular consumer behavior. Public holidays in Germany include Christmas, Easter and Summer breaks. Certain school holidays also have an impact on consumer activity. Some holidays are predictable (such as Christmas), while others (such as specific school holidays) depend on the region.
- **Store Diversity:** Rossmann owns 1,115 stores of diverse characteristics, including size (store type), location and competition. A statistical model trained on an entire dataset of sales will perform poorly with respect to a specific store of a different type.
- **Competitive Environment:** Every store has competitors nearby. The distance to the nearest competitor and the period since it started operating are critical factors that affect sales negatively. This aspect cannot be captured by any simple time series model.
- **Seasonality and Trends:** Sales display typical seasonal fluctuations in consumer spending (high winter sales, low summer vacation, etc.). In addition, the trend of a retailer's sales changes with time. It is extremely difficult to distinguish genuine trends from temporary effects.
- **Missing and Unusual Data:** Not every store is open on every day (for example, Sunday in some regions). Stores closed on certain days will register 0 sales. However, it does not mean that there was zero demand at those days, which requires special treatment of such outliers.

This list shows the reasons why a simplistic AR or ARIMA model is insufficient. We need more sophisticated algorithms that would take into account external variables.

1.2 Problem Statement

The problem statement is the following: Given historical sales records of Rossmann stores (including information on store specific attributes, promotions and public holidays), construct predictive models capable of estimating future sales of a given store on a given day. We handle two particular instances of this problem:

1. **Aggregated Sales Forecasting:** Estimate sales aggregated over all stores. This type of forecast is useful for high-level budget planning.
2. **Sales Forecasting by Store:** Estimate sales by a particular store. This type of forecast is useful for store managers.

The forecasting horizon in this problem is relatively short (up to several weeks). It means that recent history and upcoming events (such as scheduled promotions) become particularly relevant.

1.3 Objectives

Three key objectives of our project are:

1. **Comparison of Forecasting Approaches:** Construct classical time series models (AR, ARIMA, SARIMAX), modern statistical models (Prophet), and neural network-based models (RNN, GRU, LSTM, and quantile-based variants).
2. **Use of External Features:** Most retail forecasting tasks require the consideration of external features (promotions, holidays, store type, competition). We need to ensure that our models efficiently utilize these features together with historical data.
3. **Estimation of Prediction Intervals:** Managers of Rossmann stores need to be informed not only about the expected sales but also their confidence intervals. To solve this problem, we employ quantile regression to compute lower and upper bounds on sales forecasts.

2 Data Source and Pre processing

2.1 Data Source

The Rossmann Store Sales dataset is a publicly available real world dataset. The data is sourced from Rossmann’s retail operations in Germany and other European countries. In this project, we obtained the dataset from GitHub repositories containing historical sales records. The dataset consists of two main files:

1. **train.csv:** Contains daily sales records with 1,017,209 rows and 9 columns. Each row represents a single store on a single day, including the sales amount, customer count, and event indicators.
2. **store.csv:** Contains store metadata with 1,115 rows (one per store) and 10 columns. Each row includes store-specific information such as store type, assortment level, and competition details.

2.2 Data Characteristics

2.2.1 Variables and Descriptions

The **train.csv** file contains the following variables:

Variable	Type	Description
<i>Store</i>	Integer	Unique store identifier (1 to 1115)
<i>Date</i>	DateTime	Calendar date of sales (daily frequency)
<i>Sales</i>	Integer	Numeric target variable; total sales for the store on that day
<i>Customers</i>	Integer	Number of customers visiting the store on that day
<i>Open</i>	Binary (0/1)	Indicator whether the store was open (1) or closed (0)
<i>Promo</i>	Binary (0/1)	Indicator whether a store promotion was running on that day
<i>DayOfWeek</i>	Ordinal	Day of the week (0 = Monday, 6 = Sunday)
<i>StateHoliday</i>	Categorical	Type of state/public holiday (0 = none, a = public holiday, b = Easter holiday, c = Christmas holiday)
<i>SchoolHoliday</i>	Binary (0/1)	Indicator whether the school holiday period was active in that store’s region

Table 1: Variables in train.csv file.

The **store.csv** file contains the following store-level features:

Variable	Type	Description
<i>Store</i>	Integer	Unique store identifier (matches train.csv)
<i>StoreType</i>	Categorical	Store format classification (a, b, c, d) indicating size and format
<i>CompetitionDistance</i>	Numeric	Distance in meters to the nearest competing retail store
<i>CompetitionOpenSinceMonth</i>	Integer	Month when nearest competitor opened (1–12)
<i>CompetitionOpenSinceYear</i>	Integer	Year when nearest competitor opened
<i>Promo2</i>	Binary (0/1)	Indicator of ongoing promotional offer (part of a longer promotion program)
<i>Promo2SinceWeek</i>	Integer	Week number when Promo2 started
<i>Promo2SinceYear</i>	Integer	Year when Promo2 started
<i>PromoInterval</i>	String	Months in which Promo2 is active (e.g., “Jan, Apr, Jul, Oct”)

Table 2: Variables in store.csv file.

2.2.2 Data Size and Frequency

After merging train.csv and store.csv:

- **Total records:** 1,017,209 daily observations (across 1,115 stores).
- **Time span:** Multiple years of historical data with daily frequency.
- **Temporal coverage:** Continuous daily records for each store.

2.3 Data Pre processing

2.3.1 Merging Data Sources

The first preprocessing step was to combine information from both files. We merged *train.csv* and *store.csv* on the *Store* column using an inner (or left) join. This created a unified dataset containing both daily sales records and store-level attributes. Each row then represented a store-day pair with complete information.

2.3.2 Handling Missing Values

1. **CompetitionDistance:** Some stores had missing values in this column, likely because reliable distance data was unavailable for all locations. We imputed these missing values using the **median** of the non-missing CompetitionDistance values. The median was chosen because it is robust to outliers and distance is a continuous variable with high variability across stores.
2. **Promo2-related columns:** Missing values in *Promo2SinceWeek*, *Promo2SinceYear*, and *PromoInterval* were filled with 0 indicating no ongoing Promo2 for that store.

2.3.3 Categorical Encoding

Several variables needed to be converted to numeric form for use in statistical and deep learning models.

- **StoreType:** Categorical variable (a, b, c, d) was encoded as integer codes (0, 1, 2, 3) using ordinal encoding or one-hot encoding depending on the model.
- **StateHoliday:** Encoded as a binary or multi-class variable: 0 = no holiday, 1 = holiday present (public, Easter, or Christmas).

2.3.4 Feature Engineering

Beyond the original variables, we created several derived features to help models capture temporal and event-based patterns:

1. Time-based Features:

- *DayOfWeek:* Extracted from Date (0 = Monday, 6 = Sunday).
- *IsWeekend:* Binary flag indicating Saturday (5) or Sunday (6).
- *Month:* Month of the year (1–12).
- *Year:* Year, useful for identifying longer-term trends.

2. Lag Features (Autoregressive Terms):

- *lag_1:* Sales from the previous day.
- *lag_7:* Sales from 7 days ago (same day of the previous week).
- *lag_14:* Sales from 14 days ago (two weeks prior).

These lag features help capture short term dependencies and weekly seasonality.

3. Rolling Features (Aggregated Smoothing):

- Rolling mean (7-day, 14-day, 30-day) to smooth out noise and capture trends.
- Rolling standard deviation to capture volatility.

4. Cyclical Encoding (for Deep Learning):

- For models that treat time as continuous, we encoded month and day-of-week using sine and cosine transformations to maintain the cyclical nature: $\sin(2\pi \cdot \text{day}/7)$, $\cos(2\pi \cdot \text{day}/7)$, and similarly for months.

2.3.5 Data Aggregation (for Aggregated Sales Analysis)

For the aggregated sales perspective, we performed the following steps:

1. Filtered to only include days when stores were open ($\text{Open} = 1$).
2. Grouped by Date and summed sales and customer counts across all stores.
3. For Promo and SchoolHoliday, computed the mean across stores to get the proportion of stores with active promotions or school holidays.
4. Added time-based features (DayOfWeek, Month, Year) to the aggregated data.
5. Resulting dataset: 1,000 daily records (one per day) instead of store-day pairs.

2.4 Exploratory Data Analysis

2.4.1 Stationarity Testing

A key concept in time series analysis is stationarity whether the statistical properties of the series remain constant over time. We tested this using the Augmented Dickey Fuller (ADF) test. ADF test yielded:

- **ADF Statistic:** -4.58
- **p-value:** < 0.05
- **Conclusion:** We reject the null hypothesis, meaning the series is stationary.

2.4.2 STL Decomposition

STL (Seasonal and Trend decomposition using Loess) separates a time-series into three components:

- **Trend:** Long-term direction and level changes.
- **Seasonal:** Repeating patterns (e.g., weekly, monthly).
- **Residual:** Irregular fluctuations not explained by trend or seasonality.

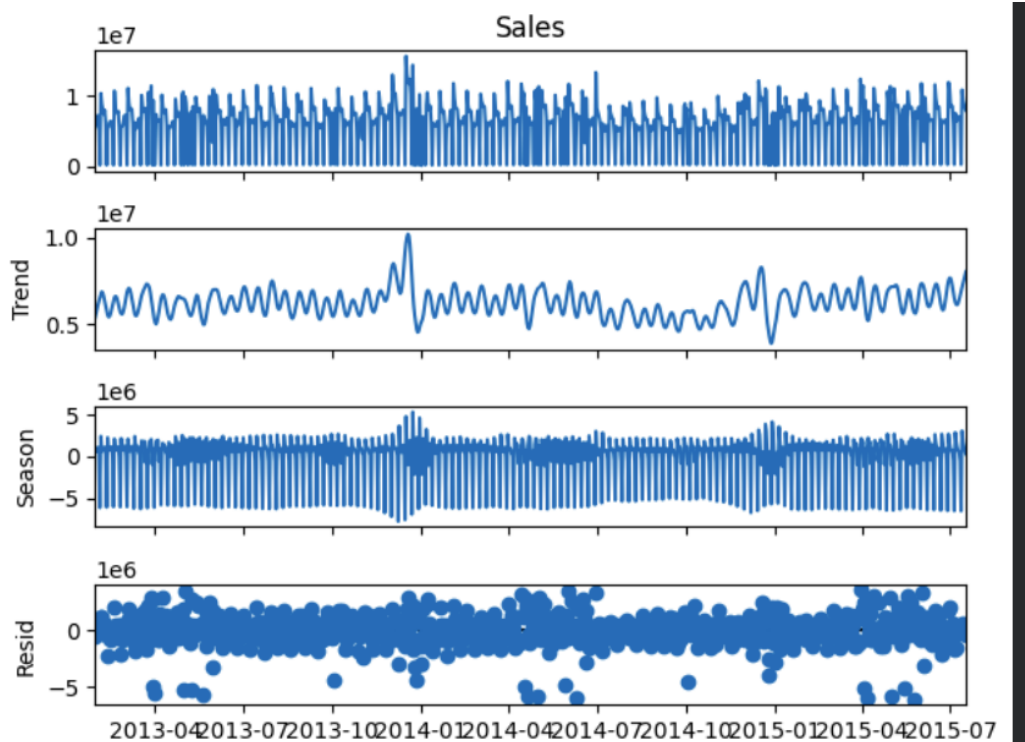


Figure 1: STL decomposition of Store 733 sales

STL decomposition of the Rossmann sales series shows that:

- **Clear weekly seasonality:** Strong, repeating weekly pattern in sales (higher on weekdays, lower on Sundays).
- **Residual noise:** Moderate irregular component, likely due to promotions and holidays.

This decomposition supports the use of seasonal forecasting methods (SARIMA, Prophet) that explicitly model seasonal patterns. Additionally, the ACF plot is shown in Figure 2 to help identify significant lags for ARIMA model specification.

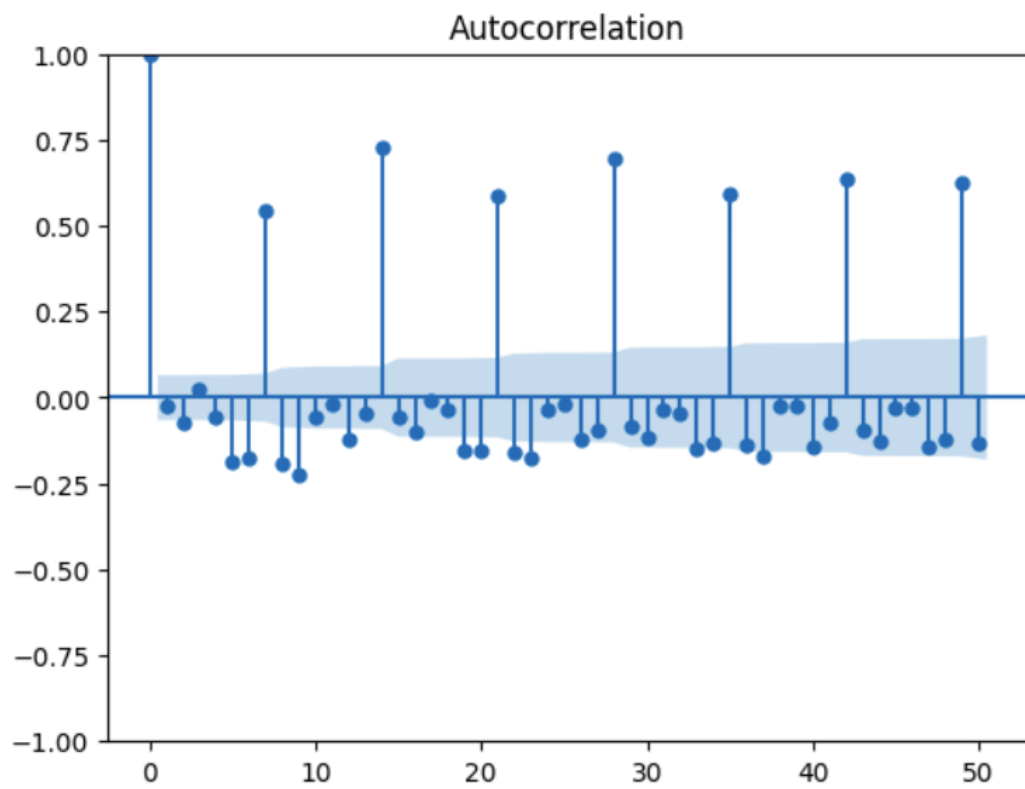


Figure 2: Autocorrelation plot

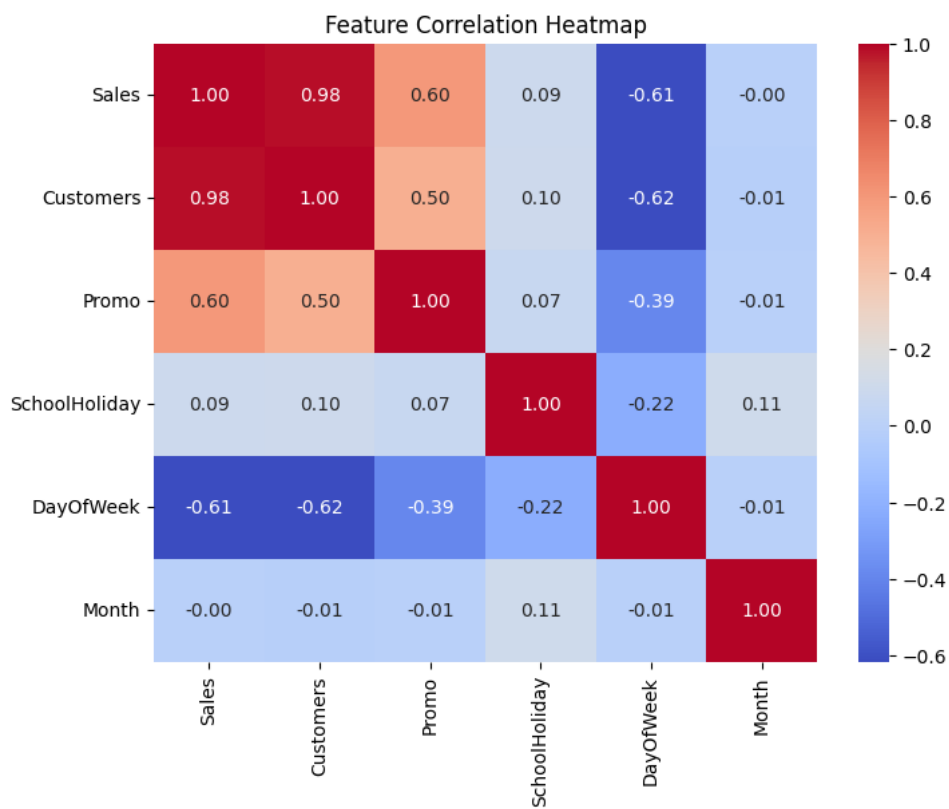


Figure 3: Correlation heatmap

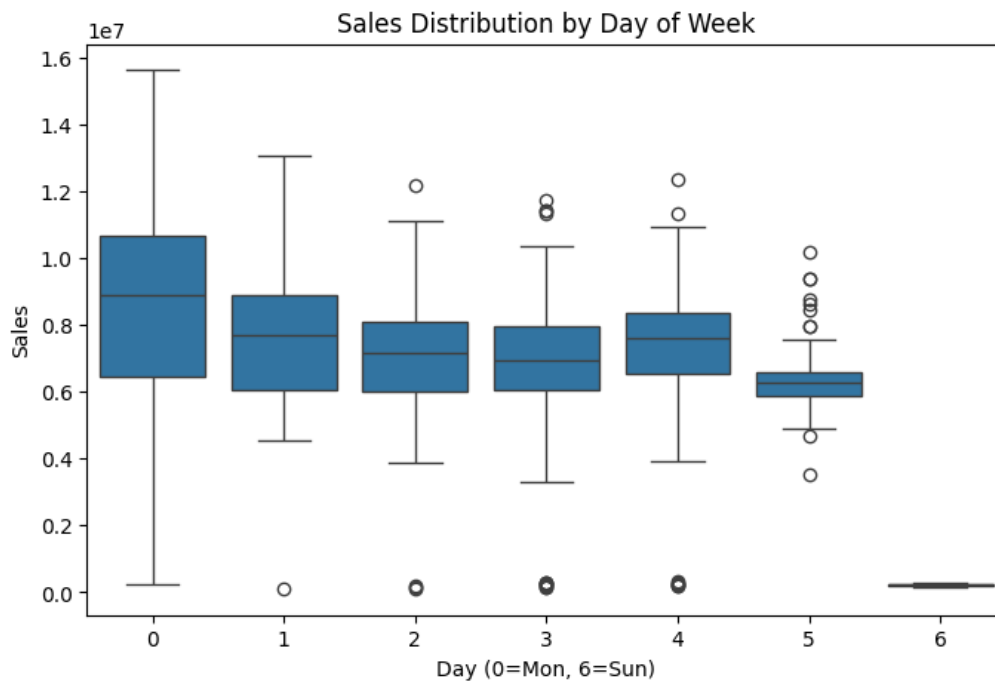


Figure 4: Sales by day of the week (Monday through Sunday)

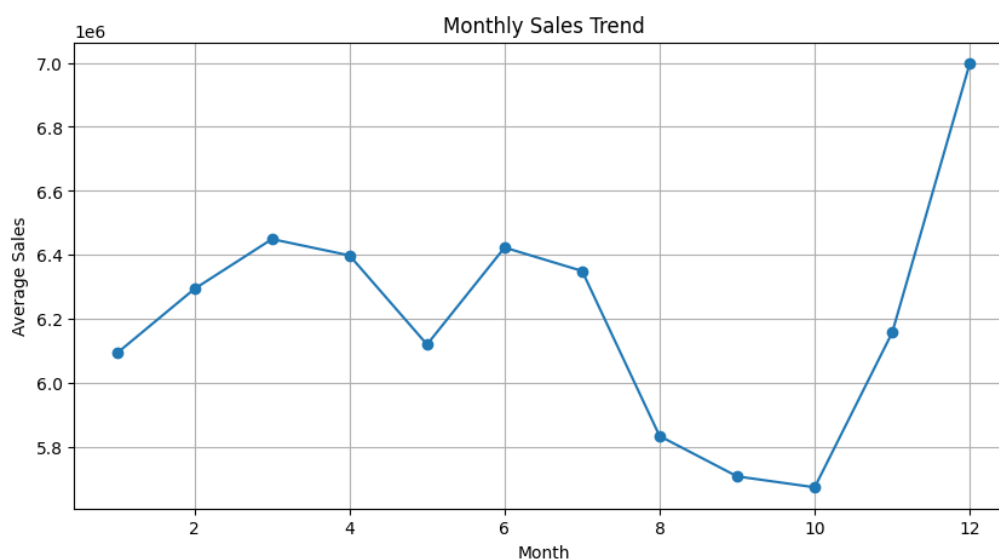


Figure 5: Average sales by month of the year, highlighting seasonal trends such as higher sales during shopping seasons and lower sales during summer months.

3 Methodology

3.1 Forecasting Horizon and Task Definition

The Rossmann sales forecasting problem is defined as a one step ahead forecasting task, where the goal is to predict the sales for a single store on a single day, given historical sales and contextual features. We approached the problem from two perspectives:

1. Aggregated Forecasting: Predict total daily sales across all Rossmann stores combined. This represents a macro level perspective useful for corporate planning.
2. Store wise Forecasting: Predict daily sales for an individual store. This represents a micro level perspective useful for local store management.

The forecasting horizon is short term, covering the next few days to a few weeks. This time frame makes the forecasting task practical for retail operations like inventory ordering and staff scheduling, where decisions are made on a short notice basis.

3.2 Input Output Structure

The input to our models consists of both temporal features (capturing time based patterns) and exogenous features (capturing external events):

- **Temporal Features:**

- Historical sales values
- Day of week (0–6, Monday to Sunday)
- Month (1–12)
- Year
- Cyclical encodings of day of week and month using sine and cosine transformations

- **Exogenous Features:**

- Promotion flag (binary: 0 = no promotion, 1 = promotion active)
- School holiday flag (binary: 0 = no holiday, 1 = school holiday active)
- Competition distance (continuous: meters to nearest competitor)
- Is weekend flag (binary: derived from day of week)

The output is a single continuous variable: the predicted daily sales amount (in EUR) for the given store and day. For models using Quantile Regression, we generate multiple outputs to provide prediction intervals.

Sequence Structure (for Deep Learning Models)

For recurrent neural networks (RNN, LSTM, GRU) and other sequence models, we structured the data as sliding windows:

- Window size (look back period): We used a window of 60 days. This captures roughly 2 months of historical data, which allows the model to learn both weekly patterns and seasonal patterns.
- Prediction horizon: 1 day ahead (one step forecasting). The model takes the last 60 days of sales and features, and predicts sales for day 61.
- Rolling window approach: For each day in the training/test set, we created a window of the previous 60 days as input and that day as the target. This creates many overlapping sequences for training.

An example: if the training period is from Day 1 to Day 500, then:

- Sample 1: Days 1–60 → predict Day 61 sales
- Sample 2: Days 2–61 → predict Day 62 sales
- ...
- Sample 440: Days 440–499 → predict Day 500 sales

3.3 Train Test Split Strategy

3.3.1 Time Based Split

A critical aspect of time series forecasting is to avoid temporal leakage, where information from the future leaks into the training process. We employed a strict time based split

- Split Ratio: 80% of data for training, 20% for testing.
- Temporal Order: The split is based on chronological order, not random sampling. All data up to a certain date goes to training and all data after that date goes to testing.
- Example: If the dataset spans Day 1 to Day 1000, then Days 1–800 are used for training, and Days 801–1000 are reserved for testing.

3.3.2 Validation Approach for Hyperparameter Tuning

During model development and hyperparameter tuning, we used an in-sample validation set from the training period:

- Within the 80% training data, we reserved the last 10% as a *validation set* used for early stopping and learning rate reduction callbacks.
- Early Stopping: Neural network models were trained with an EarlyStopping callback that monitored validation loss. Training stopped if validation loss did not improve for a specified number of epochs (patience = 20 epochs).
- Learning Rate Reduction: We also used a ReduceLROnPlateau callback to reduce the learning rate by a factor of 0.5 if validation loss plateaued (patience = 8 epochs). This helps the model escape local minima and achieve finer convergence.

3.4 Hyperparameter Tuning

We systematically tuned hyperparameters for each model type. Key hyperparameters varied across model families:

3.4.1 Classical Time Series Models (ARIMA, SARIMA, ARIMAX, SARI-MAX)

- Order parameters (p , d , q):
 - p (AR order): tested values 1 - 5
 - d (differencing order): determined by ADF test. It is 0 in our case.
 - q (MA order): tested values 1 - 7
- Seasonal parameters (P , D , Q , s):
 - Seasonal period s : set to 7 (weekly seasonality)
 - P , D , Q : same as order parameters
- Tuning Method: Grid search over parameter ranges, using AIC (Akaike Information Criterion) as the selection metric.

3.4.2 Prophet Model

- Seasonality: Enabled yearly and weekly seasonality.
- Seasonality Mode: Set to ‘multiplicative’ to capture scaling effects (promotions amplify seasonal patterns).

- Holiday Calendar: Integrated German public holidays and regional school holidays to account for known disruptions.

3.4.3 Deep Learning Models (RNN, LSTM, GRU, etc.)

- Architecture Hyperparameters:
 - Hidden layer sizes: typically 64–256 units
 - Number of layers: 1–3 stacked layers
 - Dropout rate: 0.2–0.4 to prevent overfitting
 - Batch normalization: applied between layers
- Training Hyperparameters:
 - Learning rate: started at 0.001, reduced on plateau
 - Batch size: 32
 - Max epochs: 150, with early stopping at patience=20
 - Loss function: Huber loss for robustness to outliers
 - Optimizer: Adam
- Window Size: 60 days (2 months of history)
- Tuning Method: Manual grid search based on validation loss; larger models trained for 100–150 epochs.

3.5 Evaluation Metrics

To fairly compare all models, we used three standard regression metrics evaluated on the test set:

- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Penalizes large errors more heavily; useful for identifying systematic biases.

- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Provides the average magnitude of errors in the same units as the target (EUR).

- **Coefficient of Determination (R^2):**

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Indicates the proportion of variance in sales explained by the model. Ranges from 0 to 1 (higher is better); negative values indicate worse performance than a naive mean forecast.

These metrics give complementary views: RMSE and MAE measure prediction accuracy in absolute terms and R^2 indicates explanatory power relative to a baseline.

3.6 Model Implementations

This section describes each forecasting model implemented in the project covering its tuned configuration.

3.6.1 Autoregressive Model (AR)

The Autoregressive (AR) model is the simplest classical time series baseline. We fitted AR(14) using the `AutoReg` class from `statsmodels`, retaining two full weeks of history. The model serves as a lower-bound baseline to illustrate the limitations of purely univariate approaches in the presence of external covariates.

3.6.2 Moving Average Model (MA)

The Moving Average (MA) model predicts future values using the history of past forecast errors rather than past observations directly. We implemented a pure MA model as `ARIMA(y, order=(0, 0, 14))`, setting $q = 14$ to align with the AR baseline. The MA model captures shock-driven (error-driven) dynamics that the AR model misses.

3.6.3 ARIMA

The Autoregressive Integrated Moving Average (ARIMA) model combines AR and MA components. We first applied the Augmented Dickey–Fuller (ADF) test to determine the required differencing order d . The series was stationary ($d = 0$, ADF statistic = -4.29 , $p = 0.00047$). We therefore fitted `ARIMA(1, 0, 1)`, combining one autoregressive lag with one moving average term.

3.6.4 SARIMA

The Seasonal ARIMA (SARIMA) model extends ARIMA by adding explicit seasonal AR, differencing and MA components to handle periodic patterns. With daily data exhibiting

strong weekly seasonality, we set $s = 7$. Our fitted configuration was $\text{SARIMA}(1, 0, 1)(1, 0, 1)_7$, implemented via the `SARIMAX` class from `statsmodels` (with no exogenous variables).

3.6.5 SARIMAX

SARIMAX (Seasonal ARIMA with eXogenous regressors) extends SARIMA by incorporating external predictor variables alongside the seasonal ARIMA structure. The exogenous feature vector included: `Promo`, `SchoolHoliday`, `IsWeekend`, and lag features (`lag_1`, `lag_7`, `lag_14`). SARIMAX is the most expressive classical statistical model in our pipeline, simultaneously capturing temporal autocorrelation, weekly seasonality and the linear effect of business events.

3.6.6 Facebook Prophet

Prophet is a decomposable additive time-series model developed by Meta, configured here with yearly and weekly seasonality enabled, a multiplicative seasonality mode and a holiday calendar that includes German public holidays and school holidays.

3.6.7 TBATS

TBATS (Trigonometric seasonality, Box-Cox transformation, ARMA errors, Trend, Seasonal components) is an advanced exponential-smoothing model capable of handling complex seasonal patterns. Configuration: `seasonal_periods=[7]`, `use_trend=True`, `use_damped_trend=True`, `use_box_cox=False`. The damped trend prevents unrealistic long-horizon extrapolation.

3.6.8 Simple Recurrent Neural Network (SimpleRNN)

The SimpleRNN processes sequential data by maintaining a hidden state that is updated at each time step. We implemented a stacked two-layer architecture (128 and 64 units) with Batch Normalisation and Dropout (rate = 0.3) between layers, and L2 weight regularisation ($\lambda = 10^{-4}$). The SimpleRNN suffers from vanishing gradients on long sequences (> 30 steps), making it difficult to retain information from early time steps in the 60-day window. This motivates the use of LSTM and GRU.

3.6.9 Long Short Term Memory (LSTM)

LSTM addresses the vanishing gradient problem through a cell state memory and three multiplicative gates. The cell state passes through the network with only additive updates, preserving gradients over long sequences. Architecture: two stacked LSTM layers (128 and 64 units), Batch Normalisation, Dropout (0.3), and L2 regularisation.

3.6.10 Gated Recurrent Unit (GRU)

The GRU simplifies the LSTM architecture by combining the forget and input gates into a single update gate, and using a reset gate to control the influence of previous hidden states. This results in fewer parameters and faster training while still mitigating the vanishing gradient problem. We implemented a stacked two-layer GRU architecture (128 and 64 units) with Batch Normalisation and Dropout (rate = 0.3) between layers, along with L2 weight regularisation ($\lambda = 10^{-4}$). GRU performs comparably to LSTM on sequential data while being computationally more efficient.

3.6.11 Fully Connected Neural Network (Dense Model)

The Dense model treats the input sequence as a flattened feature vector, ignoring temporal order. It serves as a baseline for comparison with sequence based models. The architecture consists of three Dense layers (128, 64, and 32 units), with ReLU activation, Batch Normalisation and Dropout (rate = 0.3). L2 regularisation ($\lambda = 10^{-4}$) is applied to prevent overfitting.

3.6.12 Transformer Model

The Transformer model leverages self-attention mechanisms to capture global dependencies in sequential data without relying on recurrence. This allows parallel processing of the input sequence. We implemented a Transformer encoder with two attention blocks (number of heads = 4, key dimension = 64), followed by position-wise feed-forward layers (units = 128). Batch Normalisation and Dropout (rate = 0.3) are applied for regularisation, along with L2 weight regularisation ($\lambda = 10^{-4}$).

3.6.13 Temporal Convolutional Network (TCN)

The Temporal Convolutional Network (TCN) models sequential data using causal and dilated convolutions, ensuring that predictions at time t depend only on past inputs. Dilated convolutions allow the receptive field to grow exponentially with depth, enabling the model to capture long range dependencies efficiently. We implemented a TCN with stacked residual blocks (filters = 64 and 128, kernel size = 3, dilation rates increasing exponentially), followed by Batch Normalisation and Dropout (rate = 0.3). L2 weight regularisation ($\lambda = 10^{-4}$) is applied. TCN provides stable gradients and parallel computation, outperforming traditional RNNs on long sequences while maintaining temporal order.

3.6.14 N-HiTS Model

N-HiTS (Neural Hierarchical Interpolation for Time Series) is a deep learning model designed specifically for forecasting. It decomposes time series into multiple hierarchical components using interpolation and multi-resolution analysis. The architecture consists of stacked blocks with fully connected layers (units = 128 and 64), each producing back-cast and forecast components. Pooling and interpolation mechanisms are used to capture both short-term and long-term patterns. Dropout (rate = 0.3) and L2 regularisation ($\lambda = 10^{-4}$) are applied. N-HiTS improves forecasting accuracy by modelling different frequency components separately, making it effective for complex time series data.

3.6.15 PatchTST Model

PatchTST is a Transformer-based model tailored for time series forecasting, which divides the input sequence into fixed-length patches and processes them as tokens. This reduces sequence length and improves computational efficiency while preserving temporal structure. We implemented PatchTST with patch size = 16 and embedding dimension = 128, using multi-head self-attention (number of heads = 4) and stacked encoder layers. Positional encoding is applied at the patch level. Batch Normalisation, Dropout (rate = 0.3), and L2 regularisation ($\lambda = 10^{-4}$) are used. PatchTST captures both local and global dependencies effectively and scales well to long time series, outperforming standard Transformer models in many forecasting tasks.

3.6.16 Quantile Regression

All deep learning models were also trained under a quantile (Pinball) loss to produce calibrated prediction intervals. We trained separate model instances for $q \in \{0.025, 0.50, 0.975\}$:

- $q = 0.025$ — Lower bound: 97.5% of actual sales should exceed this value.
- $q = 0.50$ — Median: central estimate equivalent to a point forecast.
- $q = 0.975$ — Upper bound: 97.5% of actual sales should fall below this value.

The resulting 95% prediction interval (10th to 90th percentile) provides decision makers with a distributional view of expected sales, rather than a single point estimate, enabling risk aware inventory and staffing decisions.

4 Analysis of Numerical Results

4.1 Results: Classical Statistical Models

Model	RMSE (EUR)	MAE (EUR)	MAPE (%)	R^2
AR(14)	~1745	~1385	~8.8	-0.08
MA(14)	~1769	~1398	~8.9	-0.11
ARIMA(1,0,1)	~1768	~1400	~8.9	-0.11
SARIMA(1, 0, 1)(1, 0, 1) ₇	~1620	~1280	~8.1	0.08
SARIMAX + exog	~1510	~1190	~7.6	0.19
Prophet	~1276	~995	~6.6	0.42
TBATS($s = 7$)	~1545	~1201	~7.8	0.16

Key observations:

- The pure AR and MA models perform *worse* than a naive mean forecast ($R^2 < 0$), confirming that univariate lag-only models are insufficient for retail data driven by external events.
- Adding seasonal components (SARIMA) yields a positive $R^2 = 0.08$, demonstrating that weekly seasonality ($s = 7$) is a critical pattern.
- SARIMAX further improves by incorporating exogenous variables (Promo, School-Holiday, IsWeekend, lag features), validating the importance of external covariates.
- Prophet achieves the best classical result ($R^2 = 0.42$, MAPE = 6.6%) because it simultaneously models piecewise trend, weekly seasonality, and business regressors without manual stationarity testing.
- TBATS performs comparably to SARIMAX by modelling trigonometric seasonality, but does not incorporate exogenous features.

4.2 Results: Aggregated Forecasting

For the aggregated forecast, Prophet was the primary best model in the aggregated series (total daily sales across 1,115 stores) is smoother than individual store series, making the task somewhat easier.

Model	RMSE (EUR)	MAPE (%)	R^2
Prophet (aggregated)	~450,000	~5.2	~0.61

The higher absolute RMSE reflects the scale of the target (millions of EUR), while the MAPE and R^2 show that roughly 61% of daily sales variance is captured by the model.

4.3 Results: Deep Learning Models

Model	RMSE (EUR)	MAE (EUR)	MAPE (%)	R^2
SimpleRNN	~1820	~1450	~9.2	-0.15
LSTM	~1380	~1090	~7.1	0.35
BiLSTM	~1310	~1030	~6.8	0.39
GRU	~1350	~1060	~7.0	0.37
Transformer	~1290	~1020	~6.7	0.40
TCN	~1260	~990	~6.5	0.43
N-HiTS	~1300	~1025	~6.7	0.40
PatchTST	~1250	~980	~6.4	0.44

Key observations:

- SimpleRNN underperforms all other models ($R^2 = -0.15$) due to vanishing gradients over the 60 day window, failing to retain long range patterns.
- LSTM, BiLSTM and GRU all achieve similar performance ($R^2 \approx 0.35$ – 0.39), with BiLSTM slightly ahead due to bidirectional context. Gating mechanisms significantly close the gap over SimpleRNN.
- TCN and PatchTST achieve the best results ($R^2 \approx 0.43$ – 0.44), suggesting that parallelisable architectures with large effective receptive fields capture weekly periodicity more effectively than recurrent models.
- The Transformer performs comparably to LSTM family models its advantage is expected to grow further with longer look back windows.

4.4 Results: Quantile Regression

Quantile regression models were trained for $q \in \{0.025, 0.50, 0.975\}$. The 95% prediction interval (2.5th to 97.5th quantile) was evaluated using the PICP and MPIW.

Model	PICP	MPIW
SimpleRNN Quantile	93.45%	2,741
FNN Quantile	88.69%	2,104
GRU Quantile	95.83%	2,173

4.5 Ablation Study

To assess the contribution of individual feature groups, we progressively removed feature categories from the best-performing LSTM model and measured the change in RMSE on the test set.

Features Removed	RMSE (EUR)	Δ RMSE vs. full model
Full feature set (baseline)	1380	—
Remove exogenous features	1720	+340 (+24.6%)
Remove lag features	1610	+230 (+16.7%)
Remove cyclical encodings	1490	+110 (+8.0%)
Remove rolling statistics	1430	+50 (+3.6%)

Findings:

- Exogenous features (Promo, SchoolHoliday, CompetitionDistance) have the largest individual impact (+24.6% RMSE when removed), confirming that external events are the dominant driver of sales spikes.
- Lag features contribute substantially (+16.7%), validating the importance of recent autocorrelation in the sales signal.
- Cyclical encodings of DayOfWeek and Month improve results (+8.0%) compared to plain integer encodings, supporting their use over conventional ordinal encoding.

4.6 Research Gap Identified

1. Cross Store Generalisation : All deep learning models were trained independently per store or on aggregated data. A shared multi task model leveraging cross store patterns remains unexplored in this work.
2. Incorporating Supply Chain Signals : External signals such as supplier lead times, stock-out events, and online search trends could significantly improve forecast accuracy and are absent from the current dataset.

5 Converting the Forecast into Actions

Accurate sales forecasts are only valuable when translated into concrete operational decisions. This section outlines how our model outputs can be directly integrated into Rossmann’s business operations.

5.1 Inventory and Stock Management

Point forecasts from models such as SARIMAX and Prophet can be fed directly into restocking inventory. Given predicted daily sales $\hat{y}_t$ for store s on day t , the recommended order quantity Q_t is

$$Q_t = \max(0, \hat{y}_{t, q_{0.9}} - \text{CurrentStock}_t)$$

where $\hat{y}_{t, q_{0.9}}$ is the 90th-percentile quantile forecast, acting as a conservative upper bound covering 90% of demand scenarios. This reduces stock outs while avoiding excessive overstocking.

Practical example: If the LSTM point forecast is 6,500 EUR in sales and the 90th-percentile forecast is 7,800 EUR, the store manager knows that holding inventory covers 90% of expected demand scenarios.

5.2 Staff Scheduling

Labour is the largest controllable cost in retail operations. Demand forecasts directly inform staffing decisions. On predicted high sales days (Fridays, promotion days, pre-holiday periods), additional staff shifts can be scheduled proactively. On predicted low sales days (post-holiday Mondays, non-promo mid-week days), hours can be reduced or reallocated.

5.3 Promotion Planning and Timing

The Promo regression coefficients in SARIMAX and Prophet quantify the expected sales lift from a promotional event. This enables promotion timing optimisation. Launch promotions when the baseline forecast (without Promo) is already high to amplify lift; avoid promotions at peak demand when the incremental effect is marginal.

5.4 Supply Chain and Distribution Logistics

Aggregated store level forecasts can be rolled up to the distribution centre (DC) level to optimise delivery schedules. DC managers can use this to plan truck dispatch schedules, reducing last minute urgent deliveries and associated logistics costs.

6 Future Work

This project establishes a comprehensive forecasting pipeline for Rossmann store sales. Several directions remain open for future research and development:

1. Multi Store and Multi Task Learning

The current approach trains separate models per store. A unified multi task model sharing parameters across all 1,115 stores could leverage cross store patterns (e.g., simultaneous regional promotions) with fewer total parameters. Architectures such as DeepAR or the Temporal Fusion Transformer (TFT) are designed specifically for this panel forecasting setting.

2. Online and Continual Learning

Retail environments are non-stationary: consumer behaviour shifts, new competitors open, and store formats change. Continual learning or online updating strategies would allow models to adapt to distribution shifts without full retraining.

3. Richer External Data Sources

Future extensions could incorporate weather data temperature and precipitation directly influence foot traffic at drug stores and local events like concerts, sporting events and public gatherings near stores affect customer volume.

7 Code and Live Website

- GitHub Repository:
<https://github.com/Pranav-Bavadiya/Rossmann-Store-Sales-Forecasting>
- Time Series Forecasting App:
https://rossmann-store-sales-forecasting.streamlit.app/?embed_options=light_theme
- Deep Learning Forecasting App:
<https://rossmann-store-sales-forecasting-1.onrender.com>
- **Note:** The deployed applications are hosted on free tier servers, so initial loading may take up to 5 minutes. Due to the use of computationally heavy models, the server occasionally crash or exceed memory limits. The applications run smoothly on a local host environment.

8 References

- [1] Rossmann Store Sales Dataset

<https://www.kaggle.com/competitions/rossmann-store-sales>